

AI-Assisted Coding

Staying in Control · Modul 2

Wie viel soll der Coding Agent für uns entscheiden?

120 Minuten Coding-Dojo mit FairShare

Heute ist ein Coding-Dojo

Gleiche Idee. Gleiche Teams. Gleicher Coding Agent.

Runde	Unsere Arbeitsweise
1 · One-shot	Der Agent entscheidet den Weg
2 · Slices	Wir bestimmen den nächsten Schritt
3 · Kontext + Slices	Wir definieren zusätzlich den technischen Rahmen

Jede Runde beginnt in einem **neuen, leeren Ordner und Chat**.
Wir lernen aus dem Vorgehen. Eine fertige App ist optional.

FairShare

Gemeinsame Ausgaben unter Freunden aufteilen

Die Nutzer können ...

- Personen hinzufügen und gemeinsame Ausgaben erfassen,
- angeben, wer bezahlt hat und wer beteiligt war,
- aktuelle Salden sehen,
- ermitteln, wer wem Geld zum Ausgleich zahlen sollte.

Wie die Anwendung entsteht, ist zunächst offen.

Was beobachten wir?

Dimension	Unsere Frage
Fortschritt	Welche sichtbare Funktion ist entstanden?
Verständnis	Wie gut verstehen wir den entstandenen Code?
Kontrolle	Welche wichtigen Entscheidungen haben wir selbst getroffen?
Ballast	Welche unnötige Funktion oder Komplexität kam hinzu?

Während der Runde: konkrete Beispiele notieren.
In der Reflexion: die Erfahrungen vergleichen.

Runde 1 · One-shot

15 Minuten · 00:10–00:25

Baue eine kleine Anwendung namens FairShare zum Aufteilen von Ausgaben in einer Gruppe von Freunden.

Nutzer sollen Personen hinzufügen, gemeinsame Ausgaben erfassen, Zahler und Beteiligte angeben, aktuelle Salden sehen und ermitteln, wer wem Geld zum Ausgleich zahlen sollte.

Baue eine benutzbare Version der Anwendung.

Leerer Ordner, neuer Chat. Lasst den Agenten den Weg wählen.
Echte Rückfragen beantworten. Vorerst nicht gezielt nachsteuern.

Wer hat den Weg entworfen?

Reflexion 1 · 10 Minuten

1. Was hat überraschend gut funktioniert?
2. Welche Entscheidungen hat der Agent für euch getroffen?
3. Wie sicher würdet ihr euch beim Freigeben dieses PRs fühlen?

Je größer der Auftrag, desto mehr Entscheidungen delegieren wir stillschweigend.

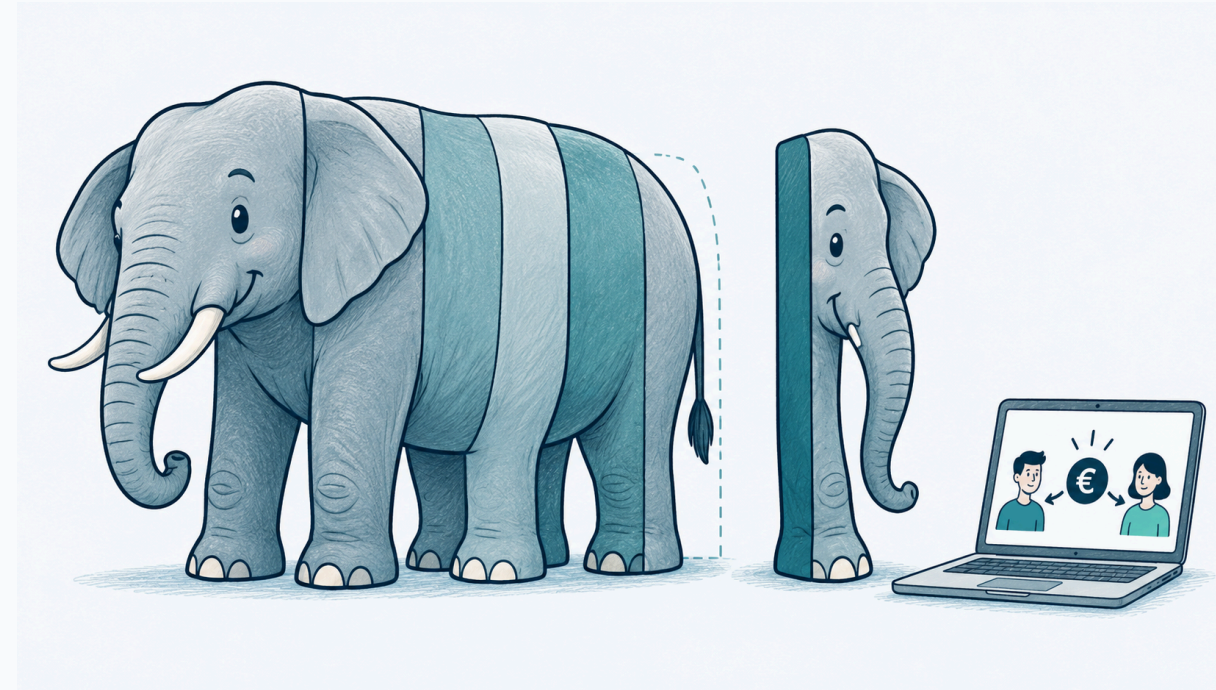
Der Elefant: „Baue FairShare“

Ein sinnvolles Produktziel.
Ein großer Auftrag an den Agenten.

Elephant Carpaccio:

kleine, nützliche vertikale Slices.

Jeder Slice zeigt bereits
ein Stück funktionierendes Verhalten.



Ein Slice zeigt nutzbares Verhalten

Technische Aufgaben	Ein vertikaler Slice
Modell erstellen	Alice bezahlt 20 € für ein Essen.
Service erstellen	Alice und Bob teilen gleichmäßig.
Oberfläche erstellen	Die App zeigt: Bob schuldet Alice 10 €.
Repository erstellen	Das Verhalten lässt sich ausführen und prüfen.

Der Slice enthält die Technik, die **dieses Verhalten** braucht.

Wie klein ist noch nützlich?

Wie klein kann der nächste Schritt sein,
sodass er noch sinnvollen Fortschritt zeigt?

„Erstelle `Expense.ts`“ beschreibt eine Datei.

„Zeige, dass Bob Alice 10 € schuldet“ beschreibt Verhalten.

Klein genug, um die Änderung zu verstehen und zu reviewen.
Groß genug, um etwas vorzuführen.

Runde 2 · Kleine nützliche Slices

20 Minuten · 00:45–01:05

Neuer leerer Ordner, neuer Chat.

Zuerst kurz eure ersten Slices festlegen.

1. Nächsten nützlichen Slice wählen.
2. Dem Agenten nur diesen Slice geben.
3. Änderung und Diff ansehen, Anwendung ausführen.
4. Änderung annehmen oder konkret nachsteuern.

Würdet ihr **diese Änderung** akzeptieren,
bevor ihr den nächsten Slice beginnt?

One-shot und Slices im Vergleich

Reflexion 2 · 10 Minuten

Dimension	Runde 1 · One-shot	Runde 2 · Slices
Fortschritt	?	?
Verständnis	?	?
Kontrolle	?	?
Ballast	?	?

Hat sich Runde 2 langsamer angefühlt?

Welche Entscheidungen hat der Agent weiterhin getroffen?

Wir bestimmen das Was

Wer bestimmt das Wie?

Entwickler: „Implementiere dieses Verhalten als Nächstes.“

Agent: „Ich wähle die technische Umsetzung.“

Slices begrenzen den nächsten Auftrag.

Technische Entscheidungen bleiben häufig offen.

Prompt und Projektkontext

Prompt	Projektkontext
Was wollen wir als Nächstes?	Wie bauen wir Software hier?
Aktueller Slice	Stabile technische Entscheidungen
Erwartetes Verhalten	Konventionen und Grenzen

Für Copilot: `.github/copilot-instructions.md`

Auch `AGENTS.md` dient vielen Agenten als Projektanweisung.
Die Unterstützung hängt vom verwendeten Tool ab.

Was gehört in den Projektkontext?

Bereich	Was das Team festlegt
Technologie	Sprache, Framework, Runtime, Datenhaltung
Architektur	Verantwortlichkeiten und ihre Grenzen
Coding Guidelines	Einfachheit, Benennung, Wiederverwendung
Agent-Grenzen	Umfang, neue Abhängigkeiten, fremde Änderungen

Relevanter, stabiler Kontext hilft bei jedem Slice.

Ein langer Wunschzettel kann die aktuelle Aufgabe verdecken.

Runde 3 · Kontext + Slices

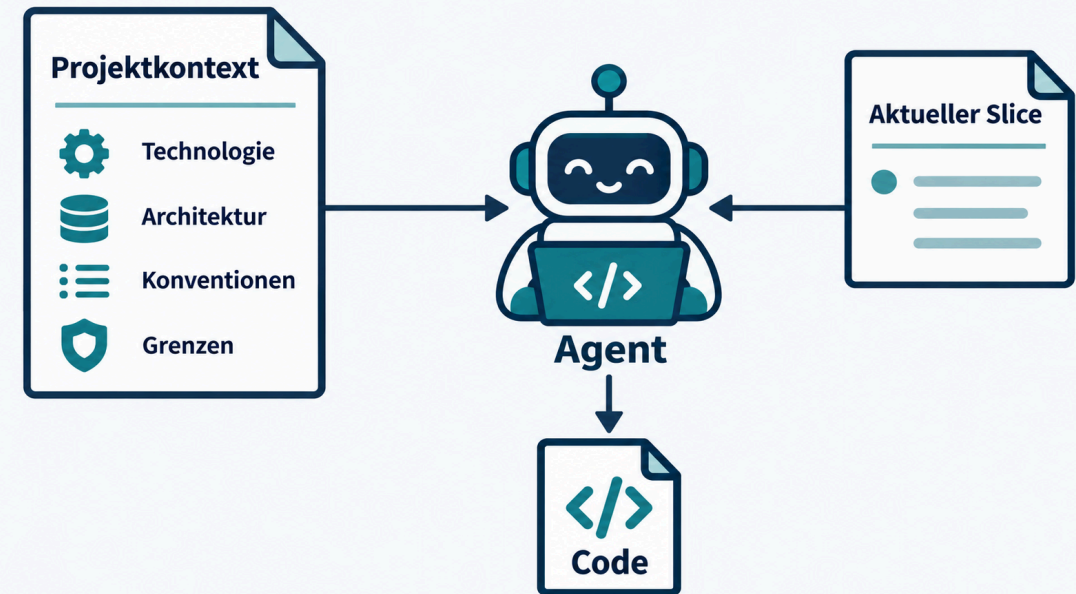
20 Minuten · 01:25–01:45

Neuer leerer Ordner, neuer Chat.

5 Minuten: Entscheidungen treffen,
Projektanweisungen schreiben.

15 Minuten: Slice wählen,
implementieren, prüfen und entscheiden.

Stabile Regeln stehen im Projekt.
Der aktuelle Auftrag steht im Prompt.



Drei Arbeitsweisen, unterschiedlich viel Kontrolle

Runde	Was wir bewusst bestimmen
1 · One-shot	Das gewünschte Ergebnis
2 · Slices	Ergebnis und nächsten Schritt
3 · Kontext + Slices	Zusätzlich den technischen Entscheidungsraum

	One-shot	Slices	Kontext + Slices
Fortschritt / Verständnis	? / ?	? / ?	? / ?
Kontrolle / Ballast	? / ?	? / ?	? / ?

Wann waren Slices zu klein oder Regeln zu eng?

Wie viel Kontrolle braucht eure konkrete Situation?

AI Code Smell Safari

5 Minuten · 01:52–01:57

Signal	Woran erkennt ihr es im eigenen Code?
Scope Creep	Eine Funktion, die niemand bestellt hat
Code Bloat	Mehr Code, als das Verhalten rechtfertigt
Vorzeitige Abstraktion	Factory, Interface oder Schicht ohne aktuellen Bedarf
Duplikation	Dieselbe Aufgabe an mehreren Stellen gelöst
Spekulative Komplexität	Aufwand für nur vorgestellte spätere Anforderungen

Ein Fund pro Team: Datei, konkrete Stelle, warum sie fragwürdig ist.

Staying in Control

1. **Die nächste Entscheidung klein halten.**
In kleinen, nützlichen Inkrementen arbeiten.
2. **Den technischen Rahmen selbst bestimmen.**
Stabile Entscheidungen als Projektkontext festhalten.
3. **Neue Änderungen weiter reviewen.**
Verhalten prüfen und den eigenen Code verstehen.

Heute: ändern, ansehen, ausführen, entscheiden.

Modul 3 · Tests as an AI Harness

Ausführbare Erwartungen machen Feedback schneller.